

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ
по дисциплине
«Введение в нереляционные системы управления базами данных»
Тема: Построитель маршрутов для раскраски карты города

Студент гр. 3388

Трунов Б.Г.

Студент гр. 3344

Гайфутдинов А.Р.

Студентка гр. 3344

Якимова Ю.А.

Преподаватель

Заславский М.М.

Санкт-Петербург

2026

ЗАДАНИЕ

Студент Трунов Б.Г. Группа 3388

Студент Гайфутдинов А.Р. Группа 3344

Студентка Якимова Ю.А. Группа 3344

Тема работы: 14. Построитель маршрутов для раскраски карты города

Исходные данные:

Реализация веб-приложения, решающего задачу оптимального покрытия городской карты треком пользователя. Пользователь движется по улицам, покрывая "тайлы"(зоны), и хочет закрасить максимальную площадь с минимальными усилиями, избегая уже закрашенных зон, приложение смотрит что уже закрашено, берёт карту улиц и строит маршрут так, чтобы нового закрасить побольше, а пройти при этом поменьше.

Содержание пояснительной записки:

«Содержание»

«Введение»

«Качественные требования к решению»

«Сценарий использования»

«Модель данных»

«Разработка приложения»

«Вывод»

«Приложение»

Предполагаемый объем пояснительной записки:

Не менее 20 страниц.

Дата выдачи задания: 13.02.2026

Дата сдачи реферата: 18.05.2026

Дата защиты реферата: 18.05.2026

Студент		Трунов Б.Г.
Студент		Гайфутдинов А.Р.
Студентка		Якимова Ю.А.
Преподаватель		Заславский М.М.

АННОТАЦИЯ

В данной курсовой работе рассматривается процесс проектирования и реализации веб-приложения для построения оптимальных маршрутов, которые помогают эффективнее покрывать "тайлы" карты во время прогулок. Основная цель работы заключается в создании системы, обеспечивающей выполнение операций создания, чтения, обновления, удаления таких сущностей как: маршруты, прогулки и точки интереса. В ходе разработки использовалась современная реляционная система управления базами данных, а именно Neo4j, для взаимодействия с картой: узлами и рёбрами, и построения маршрутов.

Особое внимание уделено реализации функционала расширенного поиска, позволяющего пользователю находить маршруты и прогулки по таким критериям как: дата и время создания/обновления, длительность, расстояние, количество покрытых "тайлов", количество точек интереса.

SUMMARY

This coursework examines the process of designing and implementing a web application for building optimal routes that help to cover the "tiles" of the map more efficiently during walks. The main goal of the work is to create a system that allows for the creation, reading, updating, and deletion of entities such as routes, walks, and points of interest. During the development process, a modern relational database management system, namely Neo4j, was used to interact with the map, including nodes and edges, and to build routes.

Special attention is paid to the implementation of advanced search functionality, which allows users to find routes and walks based on criteria such as date and time of creation/update, duration, distance, number of covered tiles, and number of points of interest.

СОДЕРЖАНИЕ

	Введение	6
1.	Выполнение задания	9
1.1.	Согласована и сформулирована тема ИДЗ	9
1.2.	Сценарии использования	9
1.2.1.	Макет пользовательского интерфейса:	10
1.2.2.	Сценарии использования.	11
1.2.3.	Преобладающий тип операций.	15
1.3.	Модель данных	15
1.3.1.	Нереляционная модель данных.	16
1.3.2.	Аналог модели данных для SQL СУБД	35
1.3.3.	Сравнение моделей	55
1.4.	Разработанное приложение	56
	Заключение	61
	ЛИТЕРАТУРА	62
	ПРИЛОЖЕНИЕ А. ДОКУМЕНТАЦИЯ ПО СБОРКЕ И РАЗВЕРТЫВАНИЮ ПРИЛОЖЕНИЯ	63
	ПРИЛОЖЕНИЕ Б	64
	ИНСТРУКЦИЯ ДЛЯ ПОЛЬЗОВАТЕЛЯ	

ВВЕДЕНИЕ

Актуальность решаемой проблемы

С развитием смартфонов и картографических сервисов прогулки перестали быть просто способом провести время на свежем воздухе. Всё больше людей увлекается идеей планомерного исследования окружающего пространства, стремясь "закрыть" как можно больше участков карты своего города. Актуальность данной работы определяется тем, что существующие навигационные приложения ориентированы на перемещение из точки А в точку Б, но не на планомерное покрытие территории. Между тем задача построения оптимального маршрута для обхода максимального числа тайлов по своей природе является графовой: улицы и перекрёстки - это рёбра и узлы, а значит, использование графовой СУБД Neo4j здесь куда органичнее, чем традиционные реляционные решения. Помимо этого, пользователю важно не только прокладывать новые маршруты, но и вести историю своих прогулок, отмечать интересные места и гибко искать нужное - по дате, длительности, пройденному расстоянию или количеству охваченных тайлов. Всё это делает проект практически востребованным и одновременно позволяет на реальной задаче изучить проектирование графоориентированных систем.

Постановка задачи

Основная цель работы заключается в создании системы, обеспечивающей выполнение операций создания, чтения, обновления, удаления таких сущностей как: маршруты, прогулки и точки интереса, используя современную реляционную систему управления базами данных, а именно Neo4j, для взаимодействия с картой: узлами и рёбрами, и построения маршрутов.

Для достижения поставленной цели необходимо решить следующие задачи:

- Разработать RESTful API для управления такими сущностями как прогулка, маршрут, точка интереса, *walkpoint*, *mapnode*, *tile*.

- Реализовать клиентскую часть для взаимодействия с API, обеспечивающую загрузку, отображение, поиск по текстовым полям и фильтрацию всех сущностей: маршруты, прогулки...
- Создать систему авторизации и регистрации пользователя.
- Обеспечить контейнеризацию всех компонентов (backend, frontend, DB,) с использованием Docker Compose для упрощения развертывания и разработки.

Предлагаемое решение

- **Архитектура и стек:** Использование клиент-серверной архитектуры. *Backend* реализован на *Python* с *FastApi*, а *Frontend* на *JavaScript*.
- **Хранение данных:** Применение нереляционной СУБД Neo4j с интеграцией через backend для эффективного управления данными.
- **Развертывание:** Развертывание всех сервисов осуществляется с помощью Docker Compose.

Качественные требования к решению

- **Безопасность:**
 - При запросах на api должна быть реализована передача token'а, чтобы пользователь не мог получать и изменять данные других пользователей.
- **Сопровождаемость:**
 - Все компоненты приложения должны быть контейнеризированы с помощью Docker Compose для обеспечения простоты развертывания и переноса между средами.
 - Код должен соответствовать принятым стандартам.
- **Удобство использования (Usability):**
 - Клиентский интерфейс должен быть интуитивно понятным, обеспечивать быструю загрузку страниц, отображение и взаимодействие с картой: функционал взятия координат(широта и долгота) точки на карте через rip-клик на карту, функционал

выделение области на карте должен быть реализован через *bbox-select*.

1. ВЫПОЛНЕНИЕ ЗАДАНИЯ

1.1. **Согласована и сформулирована тема ИДЗ**

В рамках первого этапа работы **«Согласована и сформулирована тема ИДЗ»** были выполнены следующие действия:

Согласование темы и инструментов Была выбрана и согласована тема курсовой работы: «Построитель маршрутов для раскраски карты города». Было предложено видение устройства приложения и озвучены используемые инструменты (язык программирования и СУБД).

Получение доступа к репозиторию Был получен и проверен доступ к репозиторию.

Подготовка минимального примера Для исключения проблем совместимости был подготовлен минимально работоспособный пример (скрипт для командной строки) для выбранной пары «База данных + Язык программирования». Этот скрипт подключился к БД, записал и прочитал данные.

Демонстрация и загрузка кода Работоспособность скрипта была продемонстрирована в виде скринкаста, ссылка на который отправлена преподавателю. Код тестового приложения загружен в репозиторий, в ветку main, в каталог hello_world.

1.2. **Сценарии использования**

Результаты этого этапа работы были оформлены в виде отдельной страницы на вики-репозитории под названием «Макет и сценарий использования».

Содержание вики-страницы: На странице размещены формулировки основных сценариев использования приложения в текстовом виде, а также макет пользовательского интерфейса (большое изображение).

1.2.1. Макет пользовательского интерфейса:

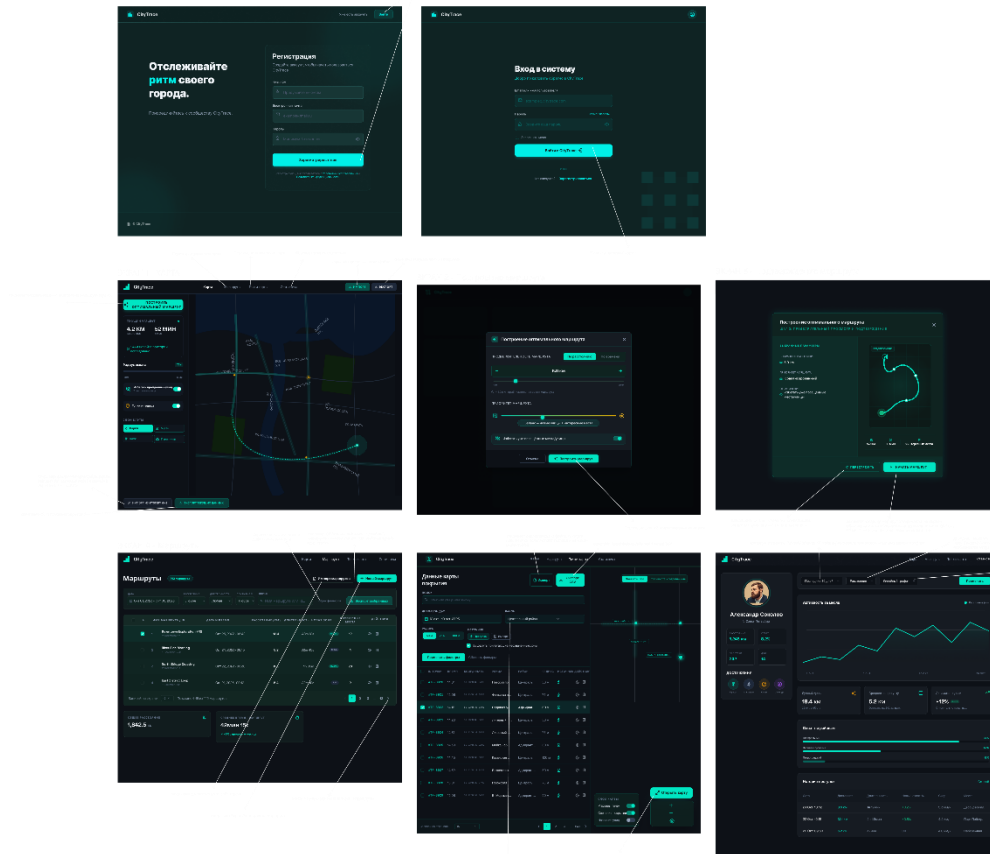


Рис №1. Макет

Макет представлен в виде графа, демонстрирующего переходы между экранами.

Файл макета загружен в репозиторий под именем `ui_mockup.png` и вставлен на вики-страницу как изображение.

Каждый экран на макете имеет уникальное название или номер, дублирование экранов отсутствует.

Все надписи и элементы на макете различимы и читаемы.

Функциональные элементы макета: Макет включает в себя элементы управления для: Массового импорта/экспорта данных. Построения кастомизируемой статистики, требования к которой описаны в задании.

Сценарии для демонстрации в прототипе:

- Первый вход
- Импорт старых треков
- Построение маршрута

- Просмотр истории маршрутов
- Работа с точками покрытия
- Статистика
- Экспорт данных

1.2.2. Сценарии использования.

Сценарий 1: Первый вход

Пользователь:

- Открывает сайт, видит лендинг
- Нажимает "Зарегистрироваться"
- Вводит имя, email, пароль, город
- Попадает на экран профиля

Сервер:

- Создает аккаунт в базе
- Загружает граф улиц для выбранного города из OpenStreetMap
- Отдает пустую карту с нулевым покрытием

Если уже есть аккаунт:

Пользователь нажимает "Войти", вводит email и пароль, попадает в профиль с уже накопленными данными.

Сценарий 2: Импорт старых треков

Когда нужно: у пользователя есть треки из Mystery Hike или GPX/CSV файл с прошлыми прогулками.

Пользователь:

- На экране Карты нажимает "Импорт из Mystery Hike" внизу или "Импорт" в навбаре
- Выбирает JSON файл
- Видит как карта обновляется -- пройденные улицы закрашиваются

Сервер:

- Парсит файл, извлекает координаты

- Сопоставляет каждую точку с узлами графа улиц
- Создаёт записи GpsPoint в Neo4j, помечает улицы как covered = true
- Отдаёт обновлённые данные покрытия на карту

Новые точки появляются в таблице Точки карты с меткой источника "Импорт"

Сценарий 3: Построение маршрута

Когда нужно: пользователь собирается на прогулку и хочет пойти по новым местам.

Шаг 1. Настройка

Пользователь:

- Нажимает "Построить оптимальный маршрут"
- Выбирает тип маршрута: кольцевой или линейный
- Выбирает значения для маршрута, например 5 км
- Двигает слайдер приоритета:
 - влево = больше новых улиц
 - вправо = больше интересных мест
 - по центру = баланс
- Нажимает "Построить маршрут"

Сервер:

- Принимает параметры: стартовая(конечная) точка, дистанция, вес приоритета
- Обходит граф улиц, исключая уже покрытые рёбра
- Учитывает POI согласно весу приоритета
- Возвращает список узлов оптимального маршрута

Шаг 2. Просмотр и подтверждение

Пользователь:

- Смотрит на миникарту с маршрутом
- Читает статистику: 4.2 км / 52 мин / 3 интересных места
- Если не нравится -- нажимает "Перестроить"

- Если устраивает -- нажимает "Начать маршрут"

Сервер:

- При "Перестроить": строит альтернативный вариант с теми же параметрами
- При "Начать маршрут": сохраняет маршрут как активный, отдаёт данные для отображения на карте

Сценарий 4: Просмотр истории маршрутов

Когда нужно: пользователь хочет посмотреть прошлые прогулки или найти конкретную.

Пользователь:

- Переходит на экран Маршруты
- Применяет фильтры по дате, расстоянию, проценту новой территории
- Нажимает иконку глаза на строке -- попадает на Карту с этим маршрутом
- Выбирает несколько маршрутов чекбоксами и экспортирует

Сервер:

- Делает выборку маршрутов по фильтрам из Neo4j
- Считает агрегаты для карточек внизу: общее расстояние, среднее время
- При клике на глаз -- отдаёт геометрию маршрута для отображения на карте
- При экспорте -- генерирует JSON из выбранных записей

Сценарий 5: Работа с точками покрытия

Когда нужно: разобраться с данными, почистить мусор от GPS-погрешностей, понять где ещё не ходил.

Пользователь:

- Переходит на экран Точки карты
- Выбирает район, источник, радиус

- Нажимает "Применить фильтры"
- Видит как таблица и мини-карта обновляются вместе
- Включает "Выделить прилегающие неохваченные зоны"
- Находит мусорную точку, нажимает корзину, удаляет
- Нажимает прицел на строке -- мини-карта центрируется на этой точке
- Нажимает "Открыть карту" -- переходит на Карту с теми же фильтрами

Сервер:

- Делает выборку по фильтрам
- При включении чекбокса неохваченных зон -- находит рёбра графа рядом с точками где covered = false
- При удалении -- убирает GpsPoint из Neo4j, пересчитывает покрытие улицы
- Отдаёт обновлённые данные на мини-карту

Сценарий 6: Статистика

Когда нужно: посмотреть на прогресс, выбрать куда идти дальше.

Пользователь:

- Переходит на экран Статистика
- Меняет период, метрику и тип графика
- Нажимает "Применить"
- Кликает на "Лучший день" -- попадает в Маршруты с фильтром по этой дате
- Кликает на район в "Охват по районам" -- попадает в Точки карты с фильтром по этому району
- Нажимает "См. всё" в истории -- попадает в Маршруты

Сервер:

- Считает агрегаты за выбранный период из Neo4j
- Группирует данные по дням для графика

- Считает процент покрытия по каждому району отдельно
- Сравнивает текущую неделю с прошлой для динамики

Сценарий 7: Экспорт данных

Пользователь выбирает что экспортировать:

- Карта / "Экспорт текущих данных" -- все точки покрытия
- Маршруты / выбрать строки / "Экспорт выбранных" -- конкретные прогулки
- Точки карты / применить фильтры / "Экспорт JSON" -- отфильтрованные точки

Сервер:

- делает выборку по переданным параметрам и отдаёт JSON файл для скачивания.

1.2.3. Преобладающий тип операций.

Возможен баланс READ/WRITE: во время создания/загрузки прогулки преобладают WRITE (непрерывная запись walkpoint's), но между прогулками пользователь активно читает: смотрит историю, статистику, карту покрытия, запрашивает маршруты.

1.3. Модель данных

Результаты этого этапа работы были оформлены в виде отдельной страницы на вики-репозитории под названием «Макет и сценарий использования»

1.3.1. Реляционная модель данных.

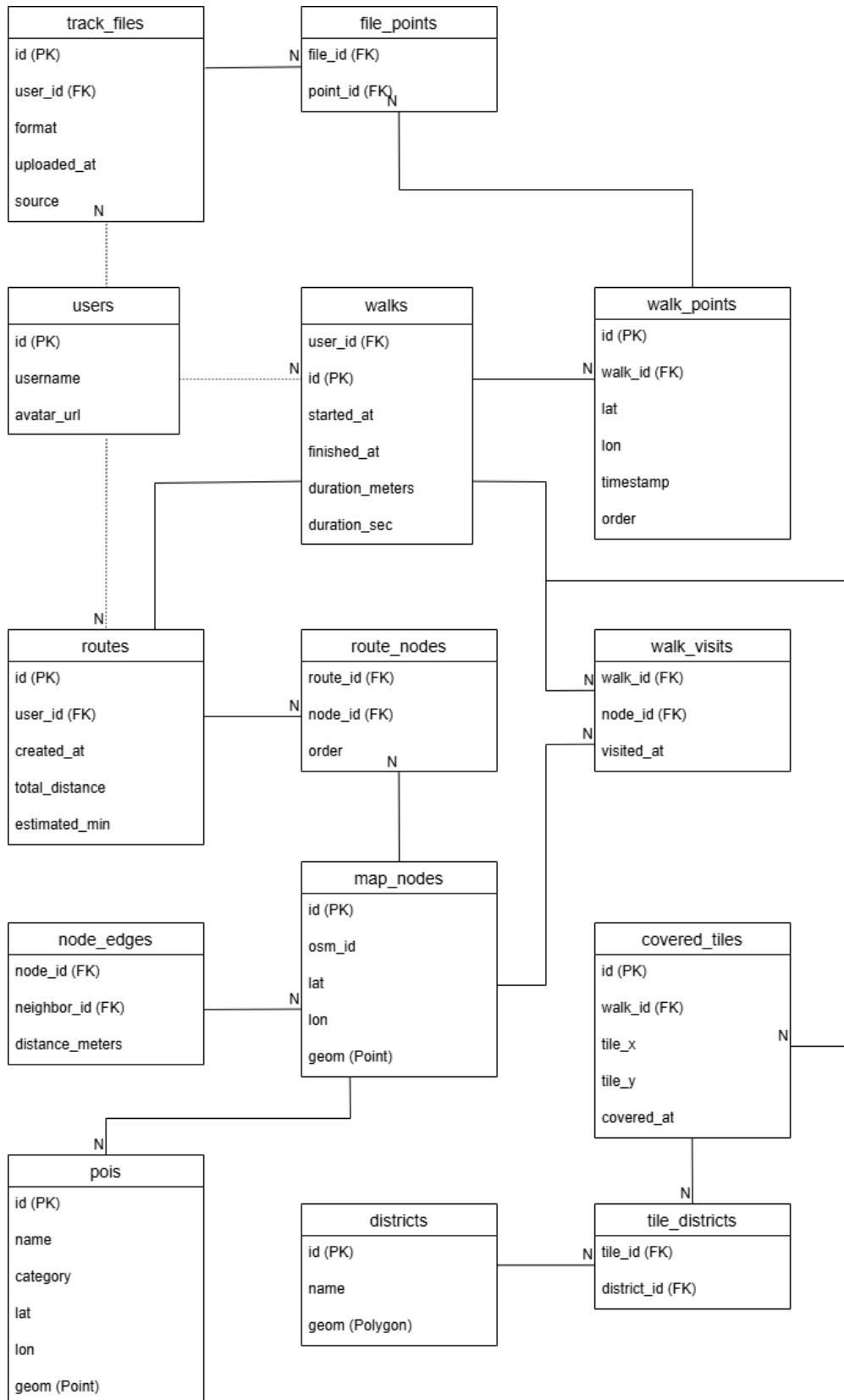


Рис №2. SQL модель

Описание таблиц, типов данных и сущностей

Общая сводка таблиц

Название	Назначение	Размер
users	Пользователи приложения	~150 б
walks	Завершённые прогулки	~60 б
walk_points	GPS-точки трека	~60 б
map_nodes	Узлы дорожного графа (OSM)	~80 б
node_edges	Рёбра графа (связи между узлами)	~40 б
walk_visits	Посещённые узлы	~40 б
covered_tiles	Закрашенные тайлы карты	~50 б
tile_districts	Связь тайлов с районами	~35 б
districts	Районы города	~560 б
pois	Точки интереса из OSM	~130 б
routes	Построенные маршруты	~55 б
route_nodes	Узлы в составе маршрута (с порядком)	~40 б
track_files	Импортированные файлы треков	~65 б
file_points	Связь файлов с GPS-точками	~35 б

users

Назначение: представляет пользователя приложения с его основной информацией профиля.

Свойство	Тип	Размер	Описание
id	UUID	16 б	Уникальный идентификатор пользователя
username	VARCHAR(50)	~50 б	Имя пользователя для отображения
avatar_url	TEXT	~80 б	URL аватара пользователя

Размер записи: ~150 б (16 + 50 + 80)

Связи

- users (1) —< walks (N): один пользователь → много прогулок
- users (1) —< routes (N): один пользователь → много маршрутов
- users (1) —< track_files (N): один пользователь → много импортированных файлов

walks

Назначение: представляет одну завершённую прогулку пользователя с основной статистикой.

Свойство	Тип	Размер	Описание
id	UUID	16 б	Уникальный идентификатор прогулки
user_id	UUID	16 б	Внешний ключ на users.id
started_at	TIMESTAMP	8 б	Время начала прогулки
finished_at	TIMESTAMP	8 б	Время окончания прогулки
distance_meters	DOUBLE	8 б	Расстояние в метрах
duration_seconds	INTEGER	4 б	Длительность в секундах

Размер записи: ~60 б

Связи

- walks (N) >— users (1): много прогулок → один пользователь
- walks (1) —< walk_points (N): одна прогулка → много GPS-точек
- walks (1) —< covered_tiles (N): одна прогулка → много закрашенных тайлов
- walks (1) —< walk_visits (N): одна прогулка → много посещённых узлов

walk_points

Назначение: представляет одну GPS-точку в треке прогулки.

Свойство	Тип	Размер	Описание
id	UUID	16 б	Уникальный идентификатор точки

Свойство	Тип	Размер	Описание
walk_id	UUID	16 б	Внешний ключ на walks.id
lat	DOUBLE	8 б	Широта
lon	DOUBLE	8 б	Долгота
timestamp	TIMESTAMP	8 б	Момент времени точки
order	INTEGER	4 б	Порядковый номер в треке

Размер записи: ~60 б

Связи

- walk_points (N) >— walks (1): много точек → одна прогулка
- walk_points (1) —< file_points (N): одна точка → много связей с файлами

map_nodes

Назначение: представляет узел дорожного графа из OpenStreetMap.

Свойство	Тип	Размер	Описание
id	UUID	16 б	Локальный идентификатор в системе
osm_id	BIGINT	8 б	Оригинальный ID из OpenStreetMap
lat	DOUBLE	8 б	Широта
lon	DOUBLE	8 б	Долгота
geom	GEOMETRY(Point, 4326)	~40 б	Пространственный тип PostGIS

Размер записи: ~80 б

Связи

- map_nodes (1) —< node_edges (N) как node_id: один узел → много исходящих рёбер
- map_nodes (1) —< node_edges (N) как neighbor_id: один узел → много входящих рёбер
- map_nodes (1) —< walk_visits (N): один узел → много посещений

- map_nodes (1) —< route_nodes (N): один узел → вхождение в много маршрутов
- map_nodes (1) —< pois (N): один узел → много POI рядом

node_edges

Назначение: представляет рёбра дорожного графа (связи между узлами OSM).

Свойство	Тип	Размер	Описание
node_id	UUID	16 б	Внешний ключ на map_nodes.id (откуда)
neighbor_id	UUID	16 б	Внешний ключ на map_nodes.id (куда)
distance_meters	DOUBLE	8 б	Расстояние между узлами

Размер записи: ~40 б

Связи

- node_edges (N) >— map_nodes (1) через node_id: много рёбер → один узел-источник
- node_edges (N) >— map_nodes (1) через neighbor_id: много рёбер → один узел-цель

walk_visits

Назначение: представляет посещение узла графа во время прогулки (аналог связи VISITS в Neo4j).

Свойство	Тип	Размер	Описание
walk_id	UUID	16 б	Внешний ключ на walks.id
node_id	UUID	16 б	Внешний ключ на map_nodes.id
visited_at	TIMESTAMP	8 б	Время посещения узла

Размер записи: ~40 б

Связи

- walk_visits (N) >— walks (1): много записей → одна прогулка

- walk_visits (N) >— map_nodes (1): много записей → один узел

covered_tiles

Назначение: представляет закрашенный тайл на карте.

Свойство	Тип	Размер	Описание
id	UUID	16 б	Уникальный идентификатор
walk_id	UUID	16 б	Внешний ключ на walks.id
tile_x	INTEGER	4 б	Х-координата тайла
tile_y	INTEGER	4 б	Y-координата тайла
covered_at	TIMESTAMP	8 б	Время закрашивания

Размер записи: ~50 б

Связи

- covered_tiles (N) >— walks (1): много тайлов → одна прогулка
- covered_tiles (1) —< tile_districts (N): один тайл → много связей с районами

tile_districts

Назначение: Связывает закрашенные тайлы с районами (аналог связи IN_DISTRICT в Neo4j).

Свойство	Тип	Размер	Описание
tile_id	UUID	16 б	Внешний ключ на covered_tiles.id
district_id	UUID	16 б	Внешний ключ на districts.id

Размер записи: ~35 б

Связи

- tile_districts (N) >— covered_tiles (1): много записей → один тайл
- tile_districts (N) >— districts (1): много записей → один район

districts

Назначение: представляет административный район города с границами.

Свойство	Тип	Размер	Описание
id	UUID	16 б	Уникальный идентификатор района
name	VARCHAR(255)	~40 б	Название района
geom	GEOMETRY(Polygon, 4326)	~500 б	Геометрия границ района

Размер записи: ~560 б

Связи

- districts (1) —< tile_districts (N): один район → много связей с тайлами

pois

Назначение: представляет точку интереса из OpenStreetMap.

Свойство	Тип	Размер	Описание
id	UUID	16 б	Уникальный идентификатор
node_id	UUID	16 б	Внешний ключ на map_nodes.id
name	VARCHAR(255)	~60 б	Название
category	VARCHAR(50)	~20 б	Категория
lat	DOUBLE	8 б	Широта
lon	DOUBLE	8 б	Долгота

Размер записи: ~130 б

Связи

- pois (N) >— map_nodes (1): много POI → один узел

routes

Назначение: представляет маршрут, построенный для пользователя.

Свойство	Тип	Размер	Описание
id	UUID	16 б	Уникальный идентификатор маршрута
user_id	UUID	16 б	Внешний ключ на users.id
created_at	TIMESTAMP	8 б	Время создания маршрута
total_distance_meters	DOUBLE	8 б	Общее расстояние маршрута
estimated_minutes	INTEGER	4 б	Ожидаемое время в минутах

Размер записи: ~55 б

Связи

- routes (N) >— users (1): много маршрутов → один пользователь
- routes (1) —< route_nodes (N): один маршрут → много узлов

route_nodes

Назначение: представляет последовательность узлов в построенном маршруте (аналог связи PASSES_THROUGH в Neo4j).

Свойство	Тип	Размер	Описание
route_id	UUID	16 б	Внешний ключ на routes.id
node_id	UUID	16 б	Внешний ключ на map_nodes.id
order_num	INTEGER	4 б	Порядковый номер узла в маршруте

Размер записи: ~40 б

Связи

- route_nodes (N) >— routes (1): много записей → один маршрут
- route_nodes (N) >— map_nodes (1): много записей → один узел

track_files

Назначение: представляет импортированный файл трека.

Свойство	Тип	Размер	Описание
id	UUID	16 б	Уникальный id файла
user_id	UUID	16 б	Внешний ключ на users.id
format	VARCHAR(10)	~10 б	Формат файла (GPX, KML)
uploaded_at	TIMESTAMP	8 б	Время загрузки
source	VARCHAR(15)	~15 б	Источник (Strava, Garmin)

Размер записи: ~65 б

Связи

- track_files (N) >— users (1): много файлов → один пользователь
- track_files (1) —< file_points (N): один файл → много связей с точками

file_points

Назначение: связывает импортированный файл с GPS-точками (аналог связи CONTAINS в Neo4j).

Свойство	Тип	Размер	Описание
file_id	UUID	16 б	Внешний ключ на track_files.id
point_id	UUID	16 б	Внешний ключ на walk_points.id

Размер записи: ~35 б

Связи

- file_points (N) >— track_files (1): много записей → один файл
- file_points (N) >— walk_points (1): много записей → одна точка

Оценка общего объема данных

Базовые допущения: введём переменную W - количество прогулок в системе. Это основная единица роста данных, так как именно прогулка порождает большинство объектов.

Примем следующие средние показатели на одну прогулку (~30 минут ходьбы):

Объект	Количество на прогулку	Обоснование
walks	1	одна запись прогулки
walk_points	500	GPS-точка каждые 3–4 секунды
covered_tiles	100	закрашенные тайлы карты
walk_visits	200	посещённые узлы (уникальные)
users	1 на 10 прогулок	один пользователь делает ~10 прогулок

Примечание: map_nodes, node_edges, districts, pois - статические данные OSM, общие для всех прогулок. Они не создаются заново при каждой прогулке и выносятся в отдельную константу C_{osm} .

Переменная часть - таблицы (на одну прогулку)

Таблица	Кол-во	Размер записи	Итого
walks	1	60 б	60 б
walk_points	500	60 б	30 000 б
covered_tiles	100	50 б	5 000 б
walk_visits	200	40 б	8 000 б
Итого			43 060 б

Дополнительно: пользователи

Один пользователь приходится на 10 прогулок:

$$W/10 \times 150 \text{ б} = W \times 15 \text{ б}$$

Итого переменная часть

$$V_{\text{var}}(W) = W \times (43\,060 + 15) = W \times 43\,060 \text{ б}$$

Постоянная часть - статические данные OSM

Постоянная часть C_{osm} - граф OSM для одного района (~5 000 узлов `map_nodes` и ~8 000 рёбер `CONNECTED_TO`):

Таблица	Кол-во	Размер записи	Итого
<code>map_nodes</code>	5 000	80 б	400 000 б
<code>node_edges</code>	8 000	40 б	320 000 б
Итого C_{osm}			720 000 б

Итоговая формула

$$V(W) = 43\,060 \times W + 720\,000 \text{ б}$$

$$V(W) \approx 42 \times W \text{ КБ} + 703 \text{ КБ}$$

Примеры при разном числе прогулок

Прогулок (W)	Переменная часть	OSM-граф	Итого
100	~4,1 МБ	0,7 МБ	~5 МБ
1 000	~41 МБ	0,7 МБ	~42 МБ
10 000	~410,1 МБ	0,7 МБ	~411 МБ

При большом W константа C_{osm} становится незначимой, и объём растёт линейно: $O(W)$.

Избыточность данных

«Чистый» объём данных - это только полезная нагрузка: координаты и время каждой GPS-точки.

Чистый объём на прогулку = `lat` (8 б) + `lon` (8 б) + `timestamp` (8 б) = $24 \text{ б} \times 500 \text{ точек} = 12\,000 \text{ б}$

$$V(W) = 43\,060 \times W + 720\,000 \text{ б}$$

$$\text{Redundancy}(W) = (43\,060 \times W + 720\,000) / (W \times 12\,000) = 3,6 + 60 / W$$

Направление роста модели

Сущность	Характер роста	Зависимость
walks, walk_points, covered_tiles, walk_visits	Растут с каждой прогулкой	$O(W)$
users	Растут медленнее прогулок	$O(W/10)$
map_nodes, node_edges, pois, districts	Статические данные OSM, растут при добавлении новых районов	$O(\text{районов})$
routes	Растут с запросами маршрутов	$O(W)$
track_files, file_points	Растут с импортами	$O(\text{импортов})$

Итоговая зависимость:

При фиксированном числе районов и N пользователях, каждый из которых совершил W прогулок:

$$V_{\text{total}} \approx 43\,060 \times (N \times W) + C_{\text{osm}} = O(N \times W)$$

При росте только числа прогулок одного пользователя - линейный рост $O(W)$. covered_tiles-узлы уникальны на пользователя, поэтому при добавлении новых пользователей объём масштабируется пропорционально их количеству.

Примеры данных

Пример 1 - Пользователь и его прогулка

```
-- Пользователь
INSERT INTO users (id, username, avatar_url)
VALUES (
    'f47ac10b-58cc-4372-a567-0e02b2c3d479',
    'alex_walker',
```

```

    'https://example.com/avatars/alex.jpg'
);

-- Прогулка
INSERT INTO walks (id, user_id, started_at, finished_at, distance_meters,
duration_seconds)
VALUES (
    'a1b2c3d4-e5f6-4a5b-c6d7-e8f9a0b1c2d3',
    'f47ac10b-58cc-4372-a567-0e02b2c3d479',
    '2026-04-15 08:30:00',
    '2026-04-15 09:15:00',
    3200.0,
    2700
);

```

Пример 2 - GPS-точки трека и закрашенный тайл

```

-- Две GPS-точки прогулки
INSERT INTO walk_points (id, walk_id, lat, lon, timestamp, order)
VALUES
(
    'wp-1111-1111-1111-111111111111',
    'a1b2c3d4-e5f6-4a5b-c6d7-e8f9a0b1c2d3',
    59.9343, 30.3351,
    '2026-04-15 08:30:15',
    1
),
(
    'wp-2222-2222-2222-222222222222',
    'a1b2c3d4-e5f6-4a5b-c6d7-e8f9a0b1c2d3',
    59.9347, 30.3358,
    '2026-04-15 08:30:19',
    2
);

-- Закрашенный тайл
INSERT INTO covered_tiles (id, walk_id, tile_x, tile_y, covered_at)
VALUES (
    'ct-aaaa-aaaa-aaaa-aaaaaaaaaaaa',
    'a1b2c3d4-e5f6-4a5b-c6d7-e8f9a0b1c2d3',
    38444, 20867,
    '2026-04-15 08:30:00'
);

```

Примеры запросов

UC-01 - Первый вход / регистрация

Создание пользователя:

```
INSERT INTO users (id, username, avatar_url)
VALUES ($1, $2, $3);
```

Параметр	Значение
Количество запросов	1
Задействованные таблицы	users
Зависимость от объёма БД	O(1)

UC-02 - Импорт треков

Создание прогулки из импортированного файла и её точек:

```
BEGIN;
```

```
-- Создание TrackFile
```

```
INSERT INTO track_files (id, user_id, format, uploaded_at, source)
VALUES ($1, $2, $3, NOW(), $4);
```

```
-- Создание Walk
```

```
INSERT INTO walks (id, user_id, started_at, finished_at, distance_meters,
duration_seconds)
VALUES ($5, $2, $6, $7, $8, $9);
```

```
-- Массовое создание WalkPoint (на каждую точку)
```

```
INSERT INTO walk_points (id, walk_id, lat, lon, timestamp, order)
VALUES ($10, $5, $11, $12, $13, $14);
```

```
-- Связь файла с точками
```

```
INSERT INTO file_points (file_id, point_id, imported_at)
VALUES ($1, $10, NOW());
```

```
COMMIT;
```

Привязка точек к узлам OSM-графа (поиск ближайшего узла):

```
-- Поиск ближайшего узла к GPS-точке
```

```
SELECT id
```

```
FROM map_nodes
ORDER BY geom <-> ST_SetSRID(ST_MakePoint($lon, $lat), 4326)
LIMIT 1;
```

Создание посещения узла:

```
INSERT INTO walk_visits (walk_id, node_id, visited_at)
VALUES ($1, $2, $3);
```

Параметр	Значение
Количество запросов	4 + P (где P - число GPS-точек)
Задействованные таблицы	track_files, walks, walk_points, file_points, map_nodes, walk_visits
Зависимость от объёма БД	$O(P \times \log)$

UC-03 - Построение маршрута

Поиск непокрытых узлов в радиусе от стартовой точки:

-- Находим стартовый узел

```
WITH start_node AS (
  SELECT id FROM map_nodes
  ORDER BY geom <-> ST_SetSRID(ST_MakePoint($startLon, $startLat), 4326)
  LIMIT 1
```

),

-- Посещённые узлы пользователя

```
visited_nodes AS (
  SELECT DISTINCT wv.node_id
  FROM walk_visits wv
  JOIN walks w ON wv.walk_id = w.id
  WHERE w.user_id = $userId
```

)

-- Построение маршрута с исключением посещённых узлов

```
SELECT
  seq,
  node,
  edge,
  cost,
  agg_cost,
  ST_AsGeoJSON(mn.geom) AS geojson
FROM pgr_dijkstra(
  $$
```

```

SELECT
    row_number() OVER () AS id,
    ne.node_id AS source,
    ne.neighbor_id AS target,
    ne.distance_meters AS cost,
    ne.distance_meters AS reverse_cost
FROM node_edges ne
WHERE ne.node_id NOT IN (SELECT node_id FROM visited_nodes)
    AND ne.neighbor_id NOT IN (SELECT node_id FROM visited_nodes)
$$,
(SELECT id FROM start_node),
$endNodeId, -- или NULL для поиска всех достижимых
false
) AS dijk
JOIN map_nodes mn ON mn.id = dijk.node
WHERE dijk.agg_cost <= $maxDistance
ORDER BY dijk.agg_cost DESC
LIMIT 1;
Сохранение построенного маршрута:

```

```

-- Создание маршрута
INSERT INTO routes (id, user_id, created_at, total_distance_meters,
estimated_minutes)
VALUES ($1, $2, NOW(), $3, $4);

```

```

-- Сохранение узлов маршрута
INSERT INTO route_nodes (route_id, node_id, order_num)
SELECT $1, unnest($5::uuid[]), unnest($6::int[]);

```

Параметр	Значение
Количество запросов	2
Задействованные таблицы	map_nodes, walk_visits, walks, node_edges, routes, route_nodes
Зависимость от объёма БД	O(N)

UC-04 - Просмотр истории прогулок

Выборка прогулок пользователя с фильтрами:

```

SELECT
    w.id,

```

```

w.started_at,
w.finished_at,
w.distance_meters,
w.duration_seconds,
COUNT(DISTINCT wp.id) AS points_count,
COUNT(DISTINCT ct.id) AS tiles_count
FROM walks w
LEFT JOIN walk_points wp ON wp.walk_id = w.id
LEFT JOIN covered_tiles ct ON ct.walk_id = w.id
WHERE w.user_id = $1
AND w.started_at >= $2
AND w.started_at <= $3
AND w.distance_meters >= $4
GROUP BY w.id
ORDER BY w.started_at DESC
LIMIT $5 OFFSET $6;

```

Геометрия одной прогулки для карты:

```

SELECT lat, lon, order
FROM walk_points
WHERE walk_id = $1
ORDER BY order ASC;

```

Параметр	Значение
Количество запросов	2
Задействованные таблицы	walks, walk_points, covered_tiles
Зависимость от объёма БД	O(W_user) для списка, O(P) для геометрии одной прогулки

UC-05 - Работа с точками покрытия

Выборка точек с фильтрами по району и источнику:

-- Точки в определённом районе

```

SELECT DISTINCT
wp.id,
wp.lat,
wp.lon,
wp.timestamp,
wp.order,
tf.source AS import_source
FROM walk_points wp

```



```

JOIN walks w ON wp.walk_id = w.id
LEFT JOIN file_points fp ON wp.id = fp.point_id
LEFT JOIN track_files tf ON fp.file_id = tf.id
JOIN covered_tiles ct ON ct.walk_id = w.id
JOIN tile_districts td ON ct.id = td.tile_id
WHERE td.district_id = $1
  AND ($2 IS NULL OR tf.source = $2)
  AND ($3 IS NULL OR ST_DWithin(
    ST_SetSRID(ST_MakePoint(wp.lon, wp.lat), 4326),
    ST_SetSRID(ST_MakePoint($4, $5), 4326),
    $6
  ))
ORDER BY wp.timestamp DESC
LIMIT $7 OFFSET $8;

```

Поиск прилегающих неохваченных зон:

```

-- Непосещённые узлы рядом с точками пользователя
SELECT DISTINCT mn.id, mn.lat, mn.lon
FROM map_nodes mn
JOIN walk_visits wv ON mn.id = wv.node_id
JOIN walks w ON wv.walk_id = w.id
WHERE w.user_id = $1
  AND NOT EXISTS (
    SELECT 1 FROM walk_visits wv2
    JOIN walks w2 ON wv2.walk_id = w2.id
    WHERE w2.user_id = $1 AND wv2.node_id = mn.id
  )
  AND ST_DWithin(
    mn.geom,
    (SELECT ST_Collect(wp.geom)
     FROM walk_points wp
     JOIN walks w3 ON wp.walk_id = w3.id
     WHERE w3.user_id = $1),
    100
  );

```

Удаление точки и пересчёт покрытия:

```

-- Начало транзакции
BEGIN;

-- Удаление точки
DELETE FROM walk_points WHERE id = $1;

-- Проверка, остались ли ещё точки на этом тайле
-- Если нет, то удаляем тайл из covered_tiles

```

```

WITH tile_info AS (
    SELECT ct.id AS tile_id, ct.walk_id
    FROM covered_tiles ct
    JOIN walk_points wp ON wp.walk_id = ct.walk_id
    WHERE ct.id = $2
    LIMIT 1
)
DELETE FROM covered_tiles
WHERE id = (SELECT tile_id FROM tile_info)
AND NOT EXISTS (
    SELECT 1 FROM walk_points wp
    JOIN covered_tiles ct ON wp.walk_id = ct.walk_id
    WHERE ct.id = (SELECT tile_id FROM tile_info)
    LIMIT 1
);

```

COMMIT;

Параметр	Значение
Количество запросов	2-3 в зависимости от операции
Задействованные таблицы	walk_points, walks, file_points, track_files, covered_tiles, tile_districts, map_nodes, walk_visits
Зависимость от объёма БД	$O(\log \text{walk_points} + K)$, где K – количество точек конкретного walk на конкретном tile.

UC-06 - Статистика

Агрегаты за период:

```

SELECT
    COUNT(*) AS total_walks,
    SUM(distance_meters) AS total_distance,
    AVG(duration_seconds) AS avg_duration,
    MAX(distance_meters) AS max_distance,
    MIN(distance_meters) AS min_distance,
    SUM(duration_seconds) AS total_duration
FROM walks
WHERE user_id = $1
    AND started_at >= $2
    AND started_at <= $3;

```

Покрытие по районам:

```

SELECT
  d.id,
  d.name,
  COUNT(DISTINCT ct.id) AS covered_tiles,
  COUNT(DISTINCT td.tile_id) AS total_tiles_in_district,
  ROUND(100.0 * COUNT(DISTINCT ct.id) / NULLIF(COUNT(DISTINCT
td.tile_id), 0), 2) AS coverage_percent
FROM districts d
LEFT JOIN tile_districts td ON d.id = td.district_id
LEFT JOIN covered_tiles ct ON td.tile_id = ct.id
LEFT JOIN walks w ON ct.walk_id = w.id AND w.user_id = $1
WHERE d.id = ANY($2::uuid[]) -- массив районов для фильтрации
GROUP BY d.id, d.name
ORDER BY coverage_percent DESC;

```

Параметр	Значение
Количество запросов	2
Задействованные таблицы	walks, districts, tile_districts, covered_tiles
Зависимость от объёма БД	$O(W_{user} \times T)$ - зависит от числа прогулок пользователя и тайлов на прогулку

1.3.2. Аналог модели данных для NoSQL СУБД Neo4j

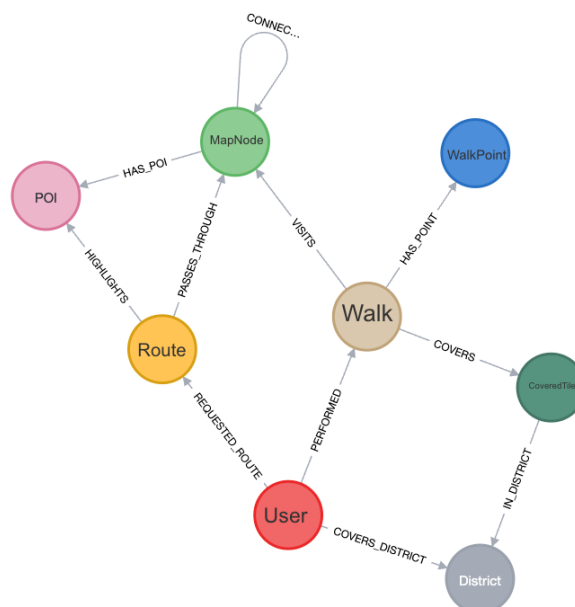


Рис. 3. NoSQL модель

Описание таблиц, типов данных и сущностей

Системы координат и переход между ними

В модели используются две согласованные системы координат:

1. Географические координаты в градусах *Применяется* в WalkPoint.lat/lon, MapNode.lat/lon, POI.lat/lon, District.polygon. Это «сырой» формат GPS и OSM.
2. **Web Mercator** - целочисленные индексы квадратных тайлов на zoom-уровне **19**. Применяется в CoveredTile.tileX/Y и в MapNode.tileX/Y. Сторона тайла **75 м**.

Формулы перехода lat/lon → tileX/tileY (zoom = 19)

$n = 2^{19} = 524288$
 $\text{tileX} = \text{floor}((\text{lon} + 180) / 360 \times n)$
 $\text{tileY} = \text{floor}((1 - \ln(\tan(\text{lat} \cdot \pi / 180) + 1 / \cos(\text{lat} \cdot \pi / 180))) / \pi) / 2 \times n)$

Правила использования

- При импорте OSM-графа для каждого MapNode один раз рассчитываются его tileX/tileY и сохраняются как свойства узла.
- После завершения или импорта прогулки для каждой WalkPoint вычисляется пара (tileX, tileY). Уникальные пары, которых ещё нет в множестве CoveredTile пользователя, создаются как новые узлы и связываются с пользователем через COVERED и с прогулкой через FIRST_COVERED.
- При построении маршрута кандидат MapNode считается «непокрытым», если пара (MapNode.tileX, MapNode.tileY) не совпадает ни с одним CoveredTile пользователя.

Узлы

Метка	Назначение	Размер
User	Пользователь приложения	~150 б
Walk	Завершённая прогулка	~50 б
WalkPoint	GPS-точка трека	~30 б
MapNode	Узел дорожного графа OSM	~40 б
CoveredTile	Закрашенный тайл карты	~20 б

Метка	Назначение	Размер
POI	Точка интереса из OSM	~115 б
District	Район города	~560 б
Route	Построенный маршрут	~40 б
TrackFile	Импортированный файл трека	~50 б

User

Назначение: представляет пользователя приложения с его основной информацией профиля.

Свойство	Тип	Размер	Описание
id	UUID	16 б	Уникальный идентификатор пользователя
username	VARCHAR(50)	~50 б	Имя пользователя для отображения
avatarUrl	String	~80 б	URL аватара пользователя

Размер записи: ~150 б (16 + 50 + 80)

Примеры использования

```
CREATE (u:User {
  id: 'f47ac10b-58cc-4372-a567-0e02b2c3d479',
  username: 'john_walker',
  avatarUrl: 'https://example.com/avatars/john.jpg'
})
```

Связи

- **PERFORMED** → Walk: прогулки пользователя
- **COVERED** → CoveredTile: тайлы, закрасенные пользователем («туман войны»)
- **REQUESTED_ROUTE** → Route: запрошенные маршруты
- **COVERS_DISTRICT** → District: Районы, закрытые пользователем

Walk

Назначение: представляет одну завершённую прогулку пользователя с основной статистикой.

Свойство	Тип	Размер	Описание
id	UUID	16 б	Уникальный идентификатор прогулки
startedAt	DateTime	8 б	Время начала прогулки
finishedAt	DateTime	8 б	Время окончания прогулки
distanceMeters	Float	8 б	Общее расстояние в метрах
durationSeconds	Int	4 б	Длительность в секундах

Размер записи: ~50 б (16 + 8 + 8 + 8 + 4)

Примеры использования

```
CREATE (w:Walk {
  id: 'a1b2c3d4-e5f6-4a5b-c6d7-e8f9a0b1c2d3',
  startedAt: datetime('2026-04-15T08:30:00Z'),
  finishedAt: datetime('2026-04-15T09:45:00Z'),
  distanceMeters: 2500.5,
  durationSeconds: 4500
})
```

Связи

- **PERFORMED** ← User: пользователь, совершивший прогулку
- **HAS_POINT** → WalkPoint: GPS-точки трека
- **FIRST_COVERED** → CoveredTile: тайлы, впервые закрашенные в этой прогулке
- **IMPORTED_FROM** → TrackFile: источник импорта (если применимо)

WalkPoint

Назначение: представляет одну GPS-точку в треке прогулки. Множество таких точек образуют полный маршрут прогулки.

Свойство	Тип	Размер	Описание
lat	Double	8 б	Широта координаты
lon	Double	8 б	Долгота координаты
timestamp	DateTime	8 б	Момент времени, когда была записана точка
order	Int	4 б	Порядковый номер точки в треке

Размер записи: ~30 б (8 + 8 + 8 + 4)

Примеры использования

```
CREATE (wp:WalkPoint {
  lat: 52.5200,
  lon: 13.4050,
  timestamp: datetime('2026-04-15T08:30:15Z'),
  order: 1
})
```

Примечания

- Точки хранятся в хронологическом порядке
- Поле order используется для восстановления последовательности маршрута
- Может быть привязана как к прогулке (Walk), так и к файлу трека (TrackFile)
- Требуется сжатия данных при долгосрочном хранении

Связи

- **HAS_POINT** ← Walk: Прогулка, к которой относится точка
- **CONTAINS** ← TrackFile: Файл, из которого импортирована точка

MapNode

Назначение: представляет узел дорожного графа из OpenStreetMap. Служит основой для маршрутизации и для сопоставления с тайлами закрашки.

Свойство	Тип	Размер	Описание
id	UUID	16 б	Локальный идентификатор в системе
osmId	Long	8 б	Оригинальный ID из OpenStreetMap
lat	Double	8 б	Широта узла (WGS84)
lon	Double	8 б	Долгота узла (WGS84)
tileX	Int	4 б	Х-индекс тайла Web Mercator, zoom 19
tileY	Int	4 б	Y-индекс тайла Web Mercator, zoom 19

Размер записи: ~48 б (16 + 8 + 8 + 8 + 4 + 4)

Поля tileX/tileY вычисляются один раз при импорте OSM-графа и позволяют быстро сопоставлять узел с закрашенными тайлами пользователя без пересчёта формул Web Mercator в каждом запросе.

Связи

- **CONNECTED_TO** ↔ MapNode: соседние узлы дорожной сети
- **HAS_POI** → POI: связанные точки интереса
- **PASSES_THROUGH** ← Route: маршруты, проходящие через узел

CoveredTile

Назначение: представляет закрашенный тайл карты для конкретного пользователя. Формирует «туман войны»: множество тайлов, открытых пользователем за всю историю прогулок. Уникален в рамках пары (User, tileX, tileY) - повторное прохождение по уже закрашенному тайлу не создаёт дубликат.

Свойство	Тип	Размер	Описание
tileX	Int	4 б	Х-индекс тайла Web Mercator, zoom 19
tileY	Int	4 б	Y-индекс тайла Web Mercator, zoom 19
firstCoveredAt	DateTime	8 б	Время первого закрашивания

Размер записи: ~16 б

Примечания

- Координаты - Web Mercator Tile Indices на zoom-уровне 19. На широте Санкт-Петербурга сторона тайла ≈ 75 м, форма - квадрат.
- Тайл считается закрашенным, если хотя бы одна WalkPoint пользователя попала внутрь его границ.
- В БД не существует двух узлов CoveredTile с одинаковыми (tileX, tileY), связанных с одним и тем же User.

Связи

- **COVERED** $\leftarrow$ User: пользователь, которому принадлежит тайл
- **FIRST_COVERED** $\leftarrow$ Walk: прогулка, в которой тайл был впервые закрашен

Пример

```
CREATE (ct:CoveredTile {
  tileX: 158121,
  tileY: 81477,
  firstCoveredAt: datetime('2026-04-15T08:35:00Z')
})
```

POI

Назначение: представляет точку интереса из OpenStreetMap (музеи, рестораны, магазины и т.д.).

Свойство	Тип	Размер	Описание
id	UUID	16 б	Уникальный идентификатор
name	String	~60 б	Название достопримечательности
category	String	~20 б	Категория (restaurant, museum и т.д.)
lat	Double	8 б	Широта
lon	Double	8 б	Долгота

Размер записи: ~115 б (16 + 60 + 20 + 8 + 8)

Примеры использования

```
CREATE (poi:POI {
```

```

id: 'poi-123',
name: 'Brandenburg Gate',
category: 'landmark',
lat: 52.5163,
lon: 13.3777
})

```

Связи

- **HAS_POI** ← MapNode: Узлы дорожной сети рядом
- **HIGHLIGHTS** ← Route: Маршруты, выделяющие POI

District

Назначение: представляет административный район города с границами в виде GeoJSON-полигона.

Свойство	Тип	Размер	Описание
id	UUID	16 б	Уникальный идентификатор района
name	String	~40 б	Название района
polygon	GeoJSON	~500 б	Геометрия границ района

Размер записи: ~560 б (16 + 40 + 500)

Примеры использования

```

CREATE (d:District {
  id: 'district-mitte',
  name: 'Mitte',
  polygon: {
    type: 'Polygon',
    coordinates: [[[13.4, 52.5], [13.5, 52.5], [13.5, 52.6], [13.4, 52.6], [13.4, 52.5]]]
  }
})

```

Связи

- **IN_DISTRICT** ← CoveredTile: Тайлы в этом районе
- **COVERS_DISTRICT** ← User: Пользователи, закрывшие район

Route

Назначение: представляет маршрут, построенный для пользователя с учётом его предпочтений и ограничений.

Свойство	Тип	Размер	Описание
id	UUID	16 б	Уникальный идентификатор маршрута
createdAt	DateTime	8 б	Время создания маршрута
totalDistanceMeters	Float	8 б	Общее расстояние маршрута
estimatedMinutes	Int	4 б	Ожидаемое время в минутах

Размер записи: ~40 б (16 + 8 + 8 + 4)

Примеры использования

```
CREATE (r:Route {
  id: 'route-abc123',
  createdAt: datetime('2026-04-15T09:00:00Z'),
  totalDistanceMeters: 5000.0,
  estimatedMinutes: 60
})
```

Связи

- **REQUESTED_ROUTE** ← User: Пользователь, запросивший маршрут
- **PASSES_THROUGH** → MapNode: Узлы, через которые проходит маршрут
- **HIGHLIGHTS** → POI: Выделенные достопримечательности

TrackFile

Назначение: представляет импортированный файл трека (GPX, KML и т.д.) с метаданными загрузки.

Свойство	Тип	Размер	Описание
id	UUID	16 б	Уникальный идентификатор файла

Свойство	Тип	Размер	Описание
format	VARCHAR(10)	~10 б	Формат файла (GPX, KML, GeoJSON и т.д.)
uploadedAt	DateTime	8 б	Время загрузки файла
source	VARCHAR(15)	~15 б	Источник (Strava, Garmin, manual и т.д.)

Размер записи: ~50 б (16 + 10 + 8 + 15)

Примеры использования

```
CREATE (tf:TrackFile {
  id: 'file-track-001',
  format: 'GPX',
  uploadedAt: datetime('2026-04-14T18:00:00Z'),
  source: 'Strava'
})
```

Связи

- **IMPORTED_FROM** ← Walk: Прогулки, импортированные из этого файла
- **CONTAINS** → WalkPoint: GPS-точки из этого файла

Оценка объема информации, хранимой в модели

Базовые допущения: введём переменную W - количество прогулок в системе. Это основная единица роста данных, так как именно прогулка порождает большинство объектов.

Примем следующие средние показатели на одну прогулку:

Объект	Количество на прогулку	Обоснование
Walk	1	одна запись прогулки
WalkPoint	500	GPS-точка каждые 3–4 секунды
CoveredTile (новые)	70	из ~100 тайлов

Объект	Количество на прогулку	Обоснование
User	1 на 10 прогулок	один пользователь делает ~10 прогулок

Примечание: MapNode, POI, District - статические данные OSM, общие для всех прогулок. Они не создаются заново при каждой прогулке и выносятся в отдельную константу C_osm.

Переменная часть - узлы (на одну прогулку)

Узел	Кол-во	Размер записи	Итого
Walk	1	50 б	50 б
WalkPoint	500	30 б	15 000 б
CoveredTile	70	16 б	1 120 б
Итого			16 170 б

Переменная часть - рёбра (на одну прогулку)

Рёбро	Кол-во	Размер	Итого
PERFORMED (User→Walk)	1	30 б	30 б
HAS_POINT (Walk→WalkPoint)	500	35 б	17 500 б
COVERED (User→CoveredTile)	70	30 б	2 100 б
FIRST_COVERED (Walk→CoveredTile)	70	30 б	2 100 б
Итого			21 730 б

Примечание: связи COVERED и FIRST_COVERED указывают на одни и те же узлы CoveredTile - COVERED от имени пользователя формирует «туман войны», а FIRST_COVERED фиксирует, в какой именно прогулке тайл был открыт впервые. Ранее существовавшая связь VISITS удалена: её роль фильтра «непокрытости» полностью перешла к CoveredTile через поля tileX/tileY у MapNode.

User-узлы

Один пользователь приходится на 10 прогулок:

$$W/10 \times 150 \text{ б} = W \times 15 \text{ б}$$

Итоговая формула

$$V(W) = W \times (\text{узлы} + \text{рёбра}) + W \times 15 + C_{\text{osm}} \\ V(W) = W \times (16\,170 + 21\,730) + W \times 15 + C_{\text{osm}} \\ V(W) \approx 37\,915 \times W + C_{\text{osm}}$$

Постоянная часть C_{osm} - граф OSM для одного района (~5 000 узлов MapNode по 48 б, ~8 000 рёбер CONNECTED_TO по 55 б, ~500 объектов POI по 115 б, ~500 связей HAS_POI по 30 б, 1 запись District по 560 б): $C_{\text{osm}} = 5000 \times 48 + 8000 \times 55 + 500 \times 115 + 500 \times 30 + 1 \times 560 = 240\,000 + 440\,000 + 57\,500 + 15\,000 + 560 \approx 753\,060 \text{ б} \approx 735 \text{ КБ}$

Итого:

$$V(W) \approx 37\,915 \times W \text{ б} + 735 \text{ КБ} \approx 37 \times W \text{ КБ} + 735 \text{ КБ}$$

Примеры при разном числе прогулок

Прогулок (W)	Переменная часть	OSM-граф	Итого
100	~3,6 МБ	0,7 МБ	~4,3 МБ
1 000	~36 МБ	0,7 МБ	~37 МБ
10 000	~362 МБ	0,7 МБ	~362 МБ

При большом W константа C_{osm} становится незначимой, и объём растёт линейно: $O(W)$.

Избыточность данных

«Чистый» объём данных - это только полезная нагрузка без графовой структуры: координаты и время каждой GPS-точки, основные поля прогулки и доля пользователя.

$$\text{GPS-точки: } 500 \times (8 + 8 + 8) = 12\,000 \text{ б}$$

$$\text{Walk-поля: } 8 + 8 + 8 + 4 = 28 \text{ б (startedAt, finishedAt, distanceMeters, durationSeconds)}$$

$$\text{User-доля: } 150 / 10 = 15 \text{ б (один пользователь на 10 прогулок)}$$

$$\text{Итого чистый объём на прогулку} = 12\,043 \text{ б}$$

Фактический объём на прогулку из формулы выше: ~37 915 б. С учётом константы C_{osm} :

$$\text{Redundancy}(W) = V(W) / (W \times 12\,043)$$

$$= (37\,915 \times W + 753\,060) / (W \times 12\,043)$$

$$= 3.15 + 62.53 / W$$

$$\approx 3.15 \text{ при } W \gg 20$$

Избыточность составляет **~3.15x** и практически не зависит от числа прогулок - это почти постоянный коэффициент. Он отражает плату за:

- хранение рёбер графа (PERFORMED, HAS_POINT, COVERED, FIRST_COVERED)
- метаданные узлов Neo4j (внутренние идентификаторы, индексы)
- производные узлы CoveredTile, вычисляемые из трека

По сравнению с предыдущей версией модели (избыточность ~3.8x) значение улучшилось за счёт удаления связи VISITS и того, что CoveredTile больше не дублируется между прогулками одного пользователя.

Направление роста модели

Сущность	Характер роста	Зависимость
Walk, WalkPoint	Растут с каждой прогулкой	$O(W)$
CoveredTile	Растут с каждой прогулкой, но с насыщением: по мере закраски района всё меньше новых тайлов создаётся на прогулку	$O(W)$ в начале, $\rightarrow O(1)$ при полной закраске
User	Растут медленнее прогулок	$O(W / 10)$
MapNode, POI, District	Статические данные OSM, растут при добавлении новых районов	$O(\text{районов})$
Route	Растут с запросами маршрутов	$O(W)$
TrackFile	Растут с импортами	$O(\text{импортов})$

Итоговая зависимость:

При фиксированном числе районов и N пользователях, каждый из которых совершил W прогулок:

$$V_{\text{total}} \approx 37\,915 \times (N \times W) b + C_{\text{osm}} = O(N \times W)$$

При росте только числа прогулок одного пользователя - линейный рост $O(W)$. CoveredTile-узлы уникальны в рамках пользователя, поэтому при добавлении новых пользователей объём масштабируется пропорционально их количеству, а не квадратично.

Примеры при разном числе прогулок

Прогулок (W)	Переменная часть	OSM-граф	Итого
100	~4,4 МБ	0,6 МБ	~5 МБ
1 000	~44 МБ	0,6 МБ	~45 МБ
10 000	~440 МБ	0,6 МБ	~441 МБ

При большом W константа C_{osm} становится незначимой, и объём растёт линейно: $O(W)$.

Примеры данных

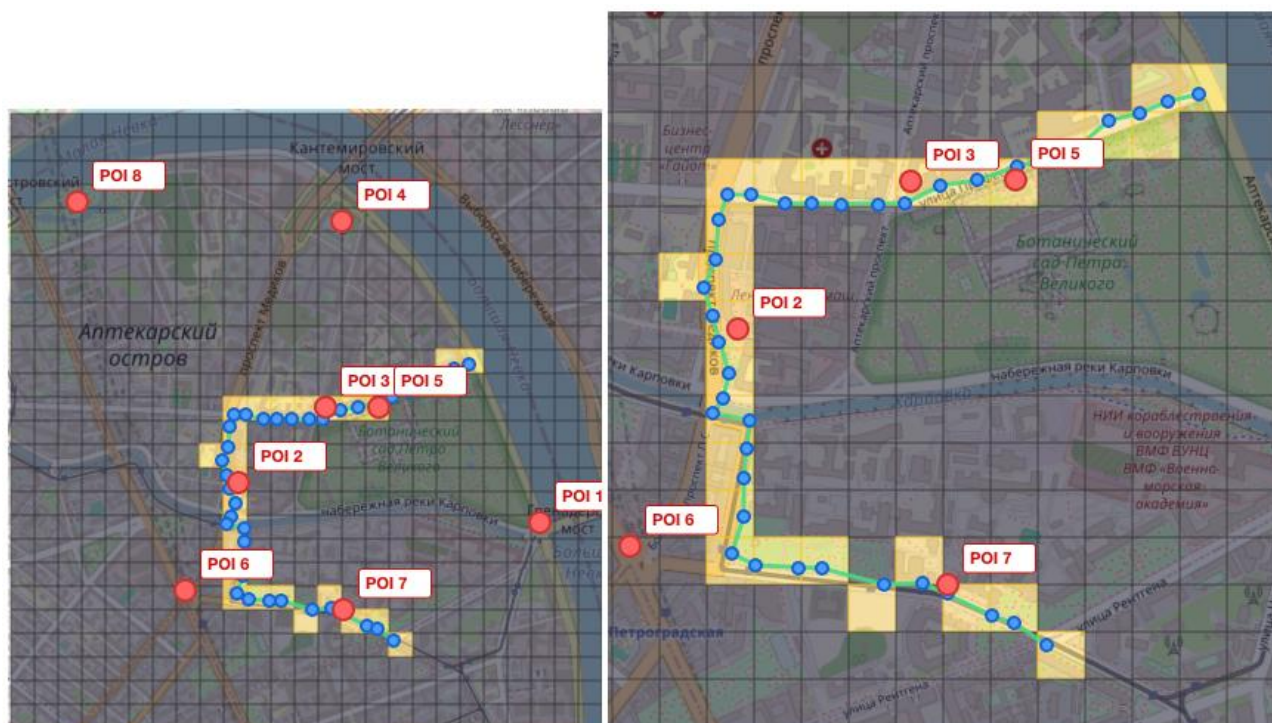


Рис. 4. Иллюстрация OSM-данных прогулки, POI, плиток закрашки (тайлы размером 75 * 75 м)

Пример 1 - Пользователь и его прогулка

```
// Пользователь
CREATE (u:User {
  id: 'f47ac10b-58cc-4372-a567-0e02b2c3d479',
  username: 'alex_walker',
  avatarUrl: 'https://example.com/avatars/alex.jpg'
```



```
)
```

```
// Прогулка
```

```
CREATE (w:Walk {  
  id: 'a1b2c3d4-e5f6-4a5b-c6d7-e8f9a0b1c2d3',  
  startedAt: datetime('2026-04-15T08:30:00Z'),  
  finishedAt: datetime('2026-04-15T09:15:00Z'),  
  distanceMeters: 3200.0,  
  durationSeconds: 2700  
})
```

```
// Связь
```

```
CREATE (u)-[:PERFORMED]->(w)
```

Пример 2 - GPS-точки и закрашенные тайлы

```
// Две GPS-точки трека
```

```
CREATE (wp1:WalkPoint {  
  lat: 59.9343, lon: 30.3351,  
  timestamp: datetime('2026-04-15T08:30:15Z'),  
  order: 1  
})
```

```
CREATE (wp2:WalkPoint {  
  lat: 59.9347, lon: 30.3358,  
  timestamp: datetime('2026-04-15T08:30:19Z'),  
  order: 2  
})
```

```
// Закрашенный тайл
```

```
CREATE (ct:CoveredTile {  
  tileX: 38444,  
  tileY: 20867,  
  coveredAt: datetime('2026-04-15T08:30:00Z')  
})
```

```
// Связи прогулки с точками и тайлом
```

```
MATCH (w:Walk {id: 'a1b2c3d4-e5f6-4a5b-c6d7-e8f9a0b1c2d3'})  
CREATE (w)-[:HAS_POINT]->(wp1)  
CREATE (w)-[:HAS_POINT]->(wp2)  
CREATE (w)-[:COVERS]->(ct)
```

Примеры запросов

UC-01 - Первый вход / регистрация

Создание пользователя:

```
CREATE (u:User {
  id: $id,
  username: $username,
  avatarUrl: $avatarUrl
})
```

Параметр	Значение
Количество запросов	1
Задействованные метки	User
Зависимость от объёма БД	O(1)

UC-02 - Импорт треков

Создание прогулки из импортированного файла и её точек:

```
// Создание TrackFile
CREATE (tf:TrackFile {
  id: $fileId,
  format: $format,
  uploadedAt: datetime(),
  source: $source
})
```

```
// Создание Walk и связь с файлом
CREATE (w:Walk {
  id: $walkId,
  startedAt: $startedAt,
  finishedAt: $finishedAt,
  distanceMeters: $distance,
  durationSeconds: $duration
})
CREATE (w)-[:IMPORTED_FROM]->(tf)
```

```
// Массовое создание WalkPoint (на каждую точку)
UNWIND $points AS p
CREATE (wp:WalkPoint {
  lat: p.lat, lon: p.lon,
  timestamp: p.timestamp,
  order: p.order
})
```

```

})
CREATE (w)-[:HAS_POINT]->(wp)
Привязка точек к узлам OSM-графа:

```

```

MATCH (mn:MapNode)
WHERE point.distance(
  point({latitude: mn.lat, longitude: mn.lon}),
  point({latitude: $lat, longitude: $lon})
) < 50
WITH mn ORDER BY point.distance(...) ASC LIMIT 1
CREATE (w)-[:VISITS]->(mn)

```

Параметр	Значение
Количество запросов	2 + P (где P - число GPS-точек)
Задействованные метки	Walk, WalkPoint, TrackFile, MapNode, CoveredTile
Зависимость от объёма БД	$O(P \times \log \text{MapNode})$ - поиск ближайших узлов

UC-03 - Построение маршрута

Назначение. Построить пешеходный маршрут от стартовой точки к финишной с максимальным закрашиванием новых тайлов пользователя.

Входные параметры:

Параметр	Описание
\$userId	ID пользователя, для которого строится маршрут
\$startLat, \$startLon	Координаты старта (обычно текущая позиция пользователя)
\$endLat, \$endLon	Координаты финиша. Для кольцевого маршрута = координатам старта
\$searchRadius	Радиус области поиска в метрах - на сколько далеко от старта разрешено удаляться (например, 1500 м)
\$targetDistance	Желаемая длина маршрута в метрах (например, 3000 м). Итоговая длина допускается в диапазоне $\pm 20\%$

Логика:

1. Найти ближайшие к старту и финишу MapNode (привязка координат к графу).

2. Среди узлов внутри \$searchRadius от старта выделить непокрытые: у которых пара (tileX, tileY) не совпадает ни с одним CoveredTile текущего пользователя.
3. Построить путь от стартового узла к финишному через граф CONNECTED_TO, максимизируя количество непокрытых узлов на пути при длине $\approx$ \$targetDistance.
4. Сохранить маршрут как Route со связями PASSES_THROUGH к узлам в порядке обхода.

Запрос 1 - привязка старта и финиша к графу:

```

MATCH (start:MapNode)
WHERE point.distance(
    point({latitude: start.lat, longitude: start.lon}),
    point({latitude: $startLat, longitude: $startLon})
) < 50
WITH start
ORDER BY point.distance(
    point({latitude: start.lat, longitude: start.lon}),
    point({latitude: $startLat, longitude: $startLon})
) ASC
LIMIT 1

```

```

MATCH (end:MapNode)
WHERE point.distance(
    point({latitude: end.lat, longitude: end.lon}),
    point({latitude: $endLat, longitude: $endLon})
) < 50
WITH start, end
ORDER BY point.distance(
    point({latitude: end.lat, longitude: end.lon}),
    point({latitude: $endLat, longitude: $endLon})
) ASC
LIMIT 1
RETURN start.id AS startNodeId, end.id AS endNodeId

```

Запрос 2 - построение маршрута с максимизацией закрашки:

```

MATCH (start:MapNode {id: $startNodeId}),
      (end:MapNode {id: $endNodeId}),
      (u:User {id: $userId})

// Все пути из старта в финиш длиной до 40 рёбер
MATCH path = (start)-[:CONNECTED_TO*1..40]-(end)

```

```

// Условие 1: все узлы пути лежат внутри радиуса поиска
WHERE ALL(n IN nodes(path) WHERE
    point.distance(
        point({latitude: start.lat, longitude: start.lon}),
        point({latitude: n.lat, longitude: n.lon})
    ) <= $searchRadius)

WITH u, path, nodes(path) AS pathNodes,
    reduce(d = 0.0, r IN relationships(path) | d + r.distanceMeters) AS totalDist

// Условие 2: итоговая длина в пределах  $\pm 20\%$  от целевой
WHERE totalDist >= $targetDistance * 0.8
    AND totalDist <= $targetDistance * 1.2

// Количество непокрытых узлов в пути:
// узел считается непокрытым, если у пользователя нет CoveredTile
// с такой же парой (tileX, tileY)
WITH path, pathNodes, totalDist,
    size([n IN pathNodes WHERE NOT EXISTS {
        MATCH (u)-[:COVERED]->(ct:CoveredTile)
        WHERE ct.tileX = n.tileX AND ct.tileY = n.tileY
    }]) AS newTiles

RETURN pathNodes AS routeNodes, totalDist, newTiles
ORDER BY newTiles DESC, abs(totalDist - $targetDistance) ASC
LIMIT 1

```

Запрос 3 - сохранение построенного маршрута:

```

MATCH (u:User {id: $userId})
CREATE (r:Route {
    id: $routeId,
    createdAt: datetime(),
    totalDistanceMeters: $distance,
    estimatedMinutes: $minutes
})
CREATE (u)-[:REQUESTED_ROUTE]->(r)

WITH r
UNWIND $nodeIds AS nodeData
MATCH (mn:MapNode {id: nodeData.id})
CREATE (r)-[:PASSES_THROUGH {order: nodeData.order}]->(mn)

```

Параметр	Значение
Количество запросов	3
Задействованные метки	MapNode, User, CoveredTile, Route
Зависимость от объёма БД	$O(\text{MapNode} \times \text{глубина_обхода})$ при фильтрации по CoveredTile пользователя

UC-04 - Просмотр истории прогулок

Выборка прогулок пользователя с фильтрами:

```

MATCH (u:User {id: $userId})-[:PERFORMED]->(w:Walk)
WHERE w.startedAt >= $dateFrom
      AND w.startedAt <= $dateTo
      AND w.distanceMeters >= $minDistance
RETURN w
ORDER BY w.startedAt DESC
SKIP $offset LIMIT $pageSize

```

Геометрия одной прогулки для карты:

```

MATCH (w:Walk {id: $walkId})-[:HAS_POINT]->(wp:WalkPoint)
RETURN wp.lat, wp.lon, wp.order
ORDER BY wp.order ASC

```

Параметр	Значение
Количество запросов	2
Задействованные метки	User, Walk, WalkPoint
Зависимость от объёма БД	$O(W_user)$ для списка, $O(P)$ для геометрии одной прогулки

UC-06 - Статистика

Агрегаты за период:

```

MATCH (u:User {id: $userId})-[:PERFORMED]->(w:Walk)
WHERE w.startedAt >= $dateFrom AND w.startedAt <= $dateTo
RETURN
  count(w) AS totalWalks,
  sum(w.distanceMeters) AS totalDistance,
  avg(w.durationSeconds) AS avgDuration,

```

max(w.distanceMeters) AS maxDistance

Покрытие по районам:

```
MATCH (u:User {id: $userId})-[:PERFORMED]->(w:Walk)-[:COVERS]->(ct:CoveredTile)-[:IN_DISTRICT]->(d:District)
```

```
RETURN d.name,
```

```
count(DISTINCT ct) AS coveredTiles
```

```
ORDER BY coveredTiles DESC
```

Параметр	Значение
Количество запросов	2
Задействованные метки	User, Walk, CoveredTile, District
Зависимость от объёма БД	$O(W_{user} \times T)$ где T - среднее число тайлов на прогулку

1.3.3 Сравнение моделей

Удельный объём информации

Метрика	Neo4j	SQL
Объём на прогулку (переменная часть)	~45 595 б	~43 060 б
Постоянная часть C_osm	~697 КБ	~703 КБ
Формула	$45\,595 \times W \text{ б} + 697 \text{ КБ}$	$43\,060 \times W \text{ б} + 703 \text{ КБ}$
Итого при W = 1 000	~45 МБ	~42 МБ
Итого при W = 10 000	~441 МБ	~411 МБ
Коэффициент избыточности	~3,8х	~3,6х

SQL компактнее на 5,8% за счёт хранения связей через FK внутри строк. В Neo4j каждое ребро - отдельная запись с накладными расходами (30–40 б). Обе модели растут линейно: $O(W)$.

Запросы по юзкейсам

Юзкейс	SQL	Neo4j
UC-01: Регистрация	1 запрос, 1 таблица, O(1)	1 запрос, 1 метка, O(1)
UC-02: Импорт треков	4 + P запросов, 6 таблиц	2 + P запросов, 5 меток
UC-03: Построение маршрута	2 запроса, 6 таблиц, требует pgRouting	2 запроса, 5 меток, нативный обход графа
UC-04: История прогулок	2 запроса, 3 таблицы	2 запроса, 3 метки
UC-05: Работа с покрытием	2–3 запроса, 8 таблиц, PostGIS	~3 запроса, 6 меток
UC-06: Статистика	2 запроса, 4 таблицы	2 запроса, 4 метки

По большинству юзкейсов Neo4j эквивалентен SQL или превосходит его. В UC-02 и UC-03 Neo4j требует меньше запросов и задействует меньше коллекций. В UC-03 принципиальное отличие: SQL вынужден использовать внешнее расширение pgRouting с JOIN-цепочкой из 6 таблиц, тогда как Neo4j выполняет обход нативно ([:CONNECTED_TO*1..20]). Единственное преимущество SQL - PostGIS для пространственных запросов (UC-05).

Вывод

Для данного приложения подходит графовая модель на основе Neo4j. Ключевые сценарии - построение маршрутов по дорожному графу и отслеживание покрытия - являются задачами обхода графа, которые Neo4j выполняет нативно и лаконично. SQL вынужден компенсировать это расширениями (pgRouting), длинными JOIN-цепочками и дополнительными join-таблицами.

1.4. Разработанное приложение

Разработанное приложение представляет собой современный полнофункциональный веб-сервис, построенный на клиент-серверной архитектуре с разделением на микросервисы, работающие в изолированных контейнерах. Бэкенд-часть написана на Python (версия 3.12 и выше) с использованием FastApi для создания высокопроизводительного API. Фронтенд функционирует как Multi Page Application, разработанный на JavaScript, с использованием Vite. Все компоненты, включая базу данных Neo4j (образ

neo4j:5.0), бэкенд, фронтенд, контейнеризированы с помощью Docker и Docker Compose.

Приложение предоставляет пользователям широкий набор функций. Основной функционал включает построение маршрутов, как кольцевых(задаём начальную точку) так и линейных(задаём конечную и начальную точки), у каждого из них перед созданием можем задать приоритет, который задаёт отношение покрытия новых территорий в будущем маршруте, к точкам интереса в нём. Для удобства работы с сущностями реализованы поиск и фильтрация по параметрам. Также реализована возможность редактирования прогулок, маршрутов, точек интереса. На экране профиля мы имеем возможность просмотра статистики за 7,30,90 дней по разным метрикам, таким как: расстояние, время, новые тайлы. Развертывание проекта описано декларативно с использованием Docker Compose. Конфигурация docker-compose.yml запускает три взаимозависимых сервиса: Neo4j, бэкенд, фронтенд. Конфиденциальные настройки передаются через переменные окружения из файла env. При первоначальном старте контейнера бэкенда автоматически выполняются миграции, создаётся тестовый пользователь и заполняются для него данные, такие как прогулки и маршруты.

Изображения приложения:

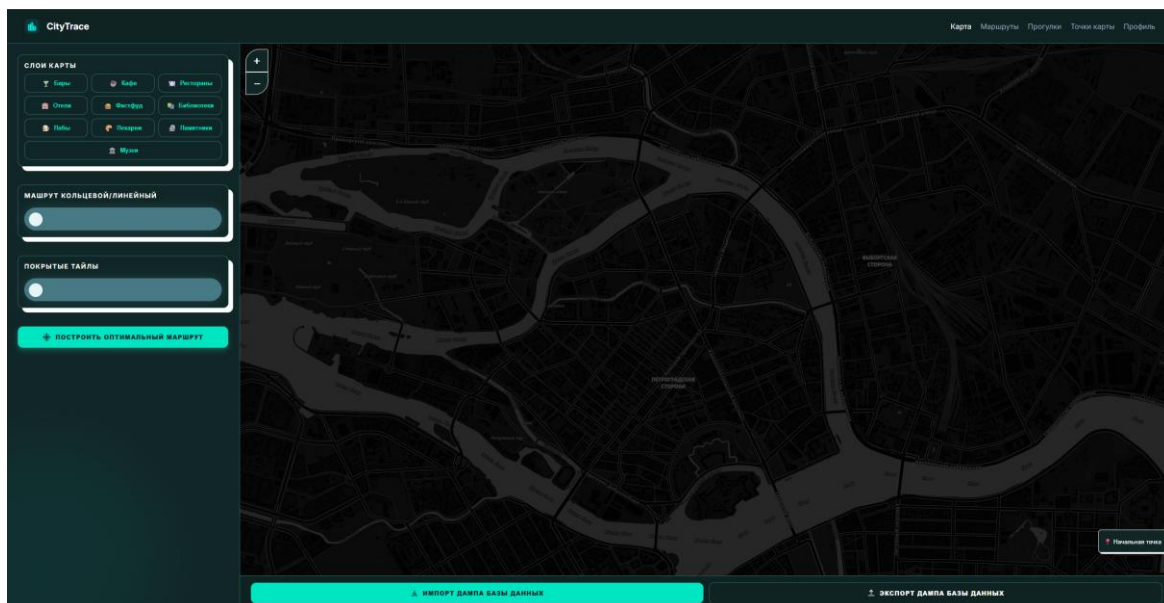


Рис. 5. Страница карты

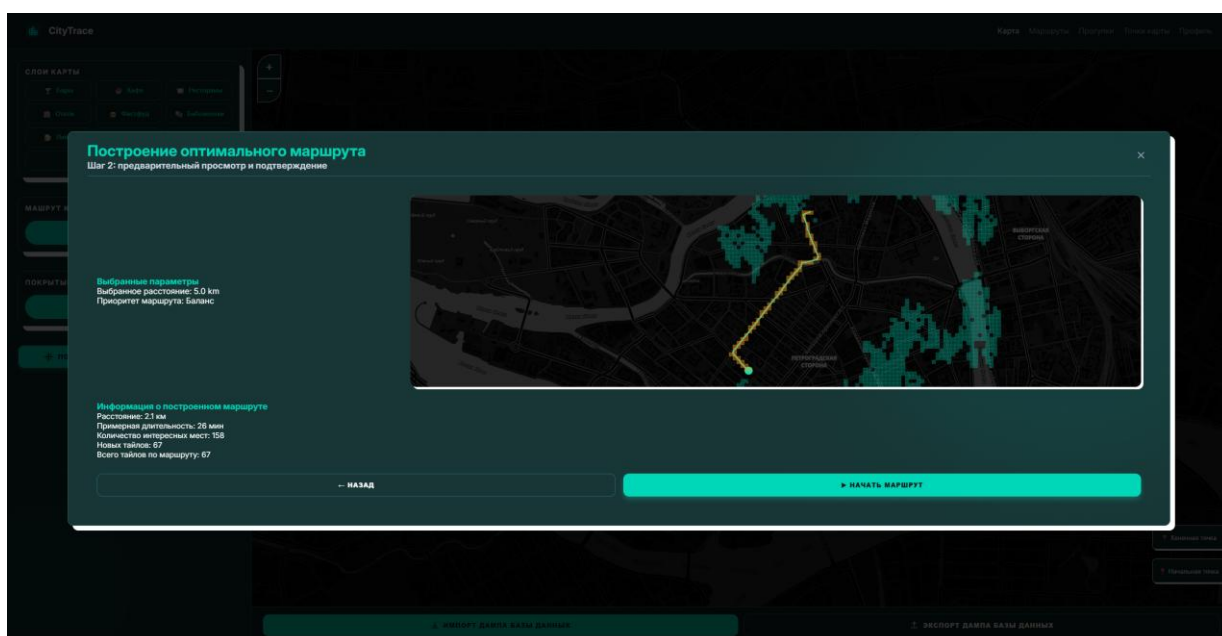


Рис. 6. Страница Карты(принятие маршрута построенного)

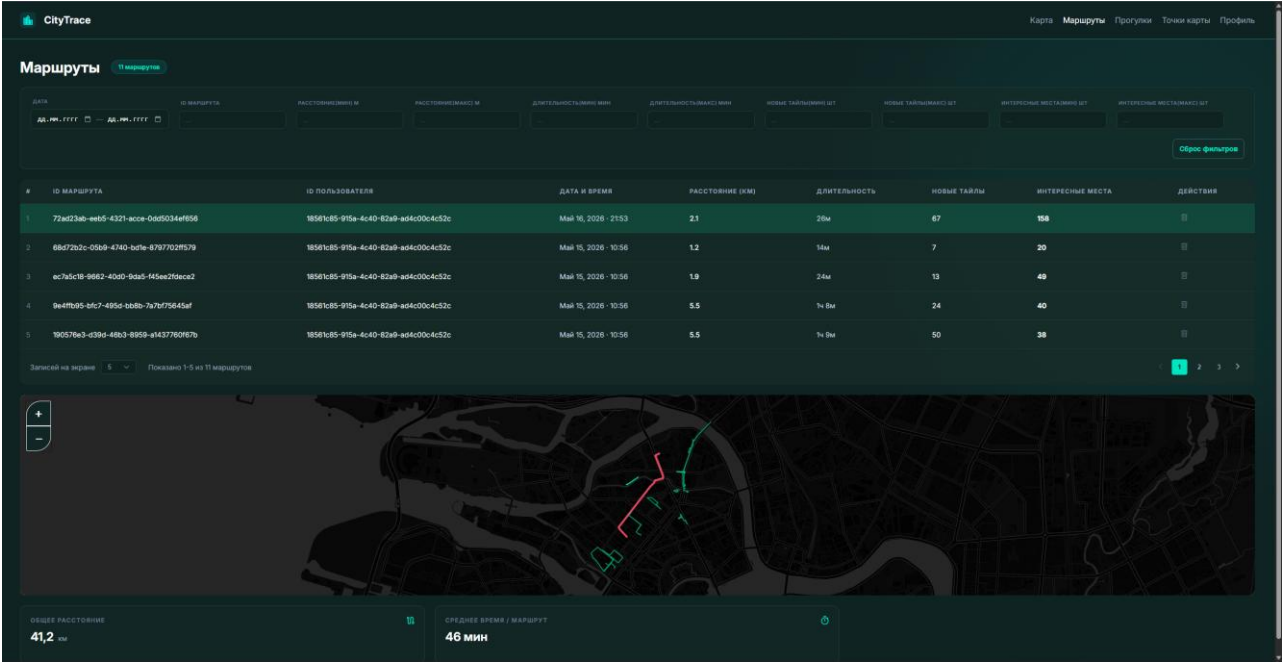


Рис. 7. Страница Маршруты

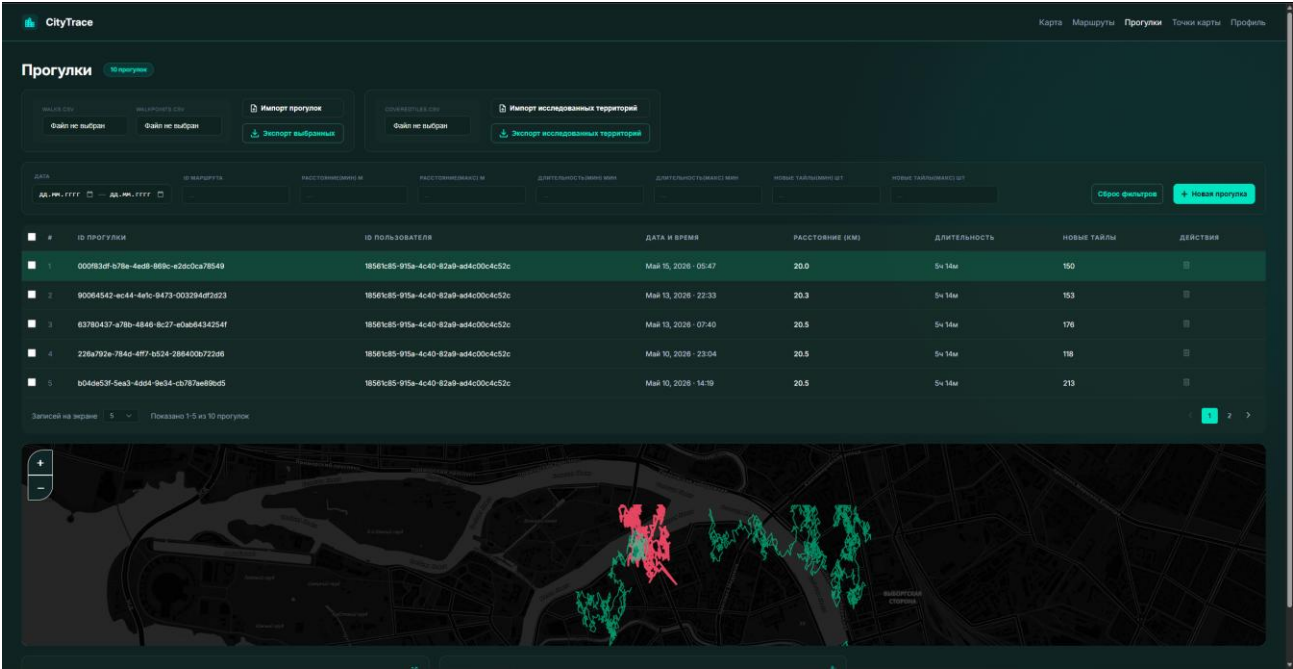


Рис. 8. Страница Прогулки

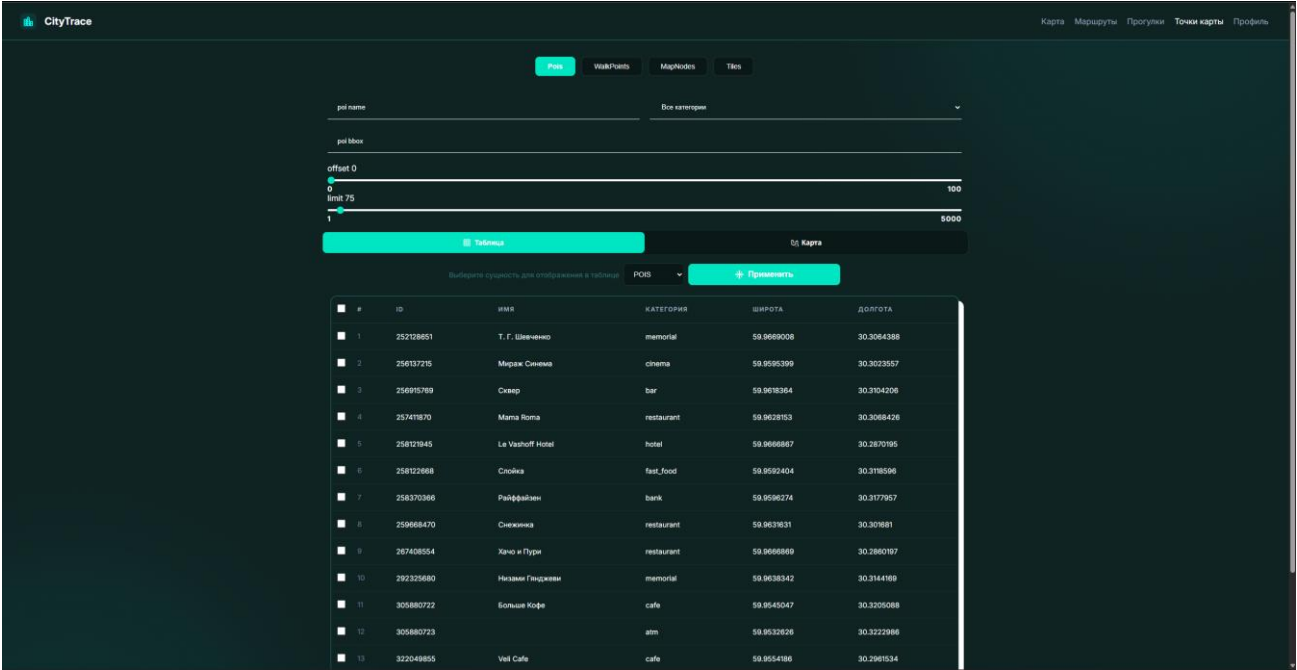


Рис. 9. Страница точки карты

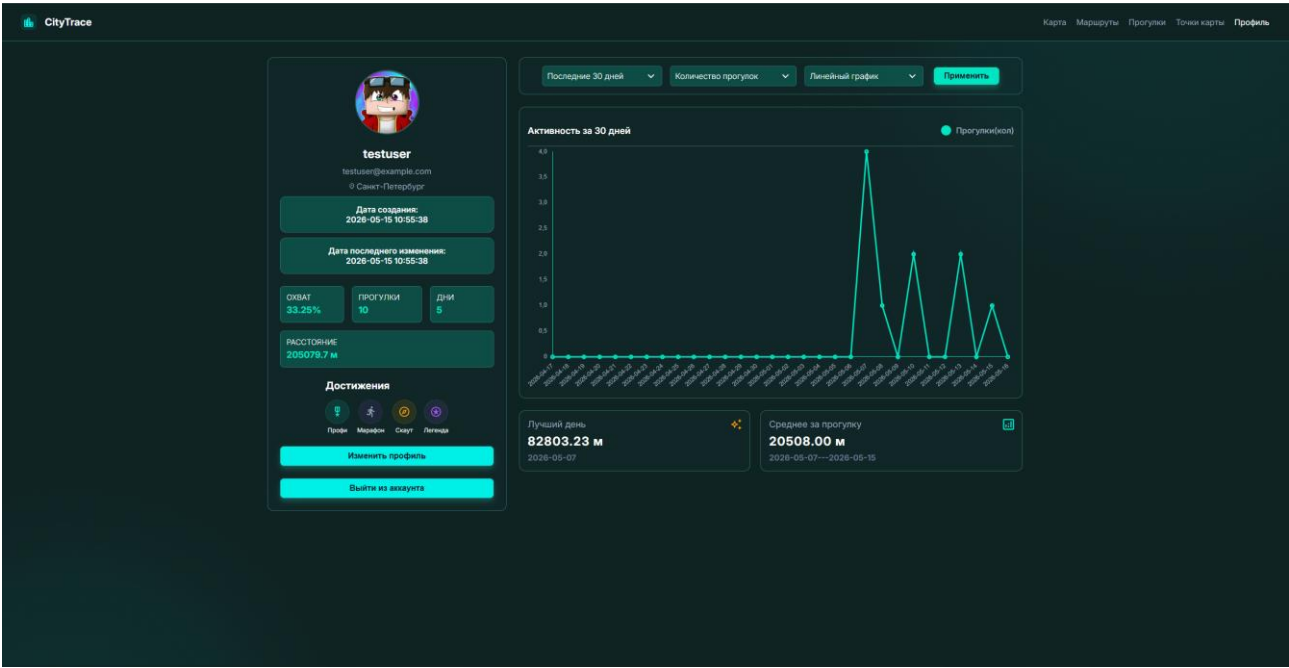


Рис. 10. Профиль пользователя

ЗАКЛЮЧЕНИЕ

Достигнутые результаты

В ходе выполнения курсовой работы была успешно решена поставленная задача по проектированию и разработке масштабируемого веб-приложения «Построитель маршрутов для раскраски карты города». Проект реализован на современном стеке технологий, включая Python, FastApi для бэкенда и JavaScript, Vite для фронтенда.

Ключевые достигнутые результаты включают: полнофункциональный CRUD для графического контента; эффективную систему поиска и фильтрации; несколько режимов построения маршрутов; а также контейнеризацию всех компонентов с помощью Docker Compose для упрощения развертывания.

Будущее развитие решения

Дальнейшее развитие проекта может заключаться в улучшении пользовательского интерфейса, адаптации к мобильным устройствам, более подробная статистика, возможность расширения приложения в социальную сеть: добавления пользователей в друзья, делиться маршрутами, смотреть процент закраски города друзей, возможность приглашать друзей на определённый маршрут, добавление геймификации в приложение: награждать пользователя за его успехи в закраске города.

ЛИТЕРАТУРА

1. Github репозиторий проекта // Github. URL:
<https://github.com/moevm/nsql1h26-map/>
2. Документация Docker и Docker-compose // Docker. URL:
<https://docs.docker.com/get-started/>
3. Документация Neo4j // Neo4j URL: <https://neo4j.com/docs/>

ПРИЛОЖЕНИЕ А

ДОКУМЕНТАЦИЯ ПО СБОРКЕ И РАЗВЕРТЫВАНИЮ ПРИЛОЖЕНИЯ

Для развертывания проекта необходимо клонировать репозиторий проекта.

```
git clone https://github.com/moevm/nsql1h26-map.git
```

Далее перейти в папку клонированного репозитория и запустить docker compose файл:

```
docker compose build --no-cache && docker compose up -d
```

После этого сайт будет доступен на <http://127.0.0.1:5678>

Данные тестового пользователя:

Логин: testuser@example.com

Пароль: test123

ПРИЛОЖЕНИЕ Б

ИНСТРУКЦИЯ ДЛЯ ПОЛЬЗОВАТЕЛЯ

Регистрация на сайте:

При первом входе на 127.0.0.1:5678 мы попадём на страницу авторизации, где в верхнем правом углу есть ссылка на страницу регистрации, на данной странице мы должны ввести никнейм, почту и пароль. После чего мы перейдём в профиль.

Авторизация на сайте:

При первом входе на 127.0.0.1:5678/ мы попадём на страницу авторизации, после чего в форме мы должны ввести логин(почту) и пароль, после чего мы перейдём в профиль.

Редактирование информации профиля:

На странице профиля(127.0.0.1:5678/profile.html) под блоком с информацией пользователя расположена кнопка редактирования информации профиля, нажимая на неё открывается модальное окно, в котором можем ввести новые данные, и сохранить их, нажав соответствующую кнопку, также изменения можно отменить просто закрыв модальное окно.

Просмотр статистики:

На странице профиля(127.0.0.1:5678/profile.html) главным блоком является блок статистики, в котором есть несколько полей с выбором: поле выбора периода, поле с выбором метрики, и кнопка применить, после нажатия на которую запрашиваемая статистика обновится на графике.

Создание маршрута:

На странице карта(127.0.0.1:5678/map.html) в левом блоке мы можем выбрать тип маршрута, который хотим построить, либо кольцевой либо линейный, после чего на карте отмечаем необходимую(ые) точку(и) и нажимаем кнопку построить маршрут в левом боковом меню, после чего откроется модальное окно, в котором мы можем выбрать характеристики маршрута, задать ему приоритет и желаемую длину с помощью соответствующих ползунков.

После выбора параметров мы нажимаем кнопку создать маршрут, и после чего мы можем либо продолжить, либо регенерировать данный маршрут.

Просмотр/редактирование маршрутов:

Всё что касается просмотра, поиска и редактирования маршрута мы делаем на странице `127.0.0.1:5678/routes.html`, при нажатии на конкретную строку в таблице мы открываем модальное окно, в котором есть возможность редактирования маршрута.

Просмотр/редактирование и добавление прогулок:

Для просмотра, поиска и редактирования и добавления прогулки мы делаем на странице `127.0.0.1:5678/walks.html`, при нажатии на конкретную строку в таблице мы открываем модальное окно, в котором есть возможность редактирования прогулки.

Для создания прогулки мы можем также нажать соответствующую кнопку, и в модальном окне при кликах на карту(таким образом мы добавляем точку прогулки) мы создаём прогулку.

Импорт экспорт дампа базы данных:

Импорт экспорт дампа базы данных происходит на экране карты(`127.0.0.1:5678/map.html`), там есть две соответствующие кнопки экспорта и импорта, при нажатии на них на странице будет идти loader, пока процесс не завершится.

Просмотр/редактирование точек интереса:

Данные действия происходят на странице `127.0.0.1:5678/points.html`, аналогичным образом со страницами `walks`, `routes`.